ENVELOPE A

TECHNICAL PROPOSAL

Cloud-Native Application Orchestration & Deployment

Submitted To

DESC Digital Innovation Center, Mardan

Tender Reference No: DESC-MRD-2026-CNC-088

Contract Duration: 1 Year (12 Months) Managed Service

Submitted By

Team DeployX

Mardan, Khyber Pakhtunkhwa, Pakistan

deployxteam@gmail.com | +92-349-9372122

Team Leader:	Nasrullah Khan
Team Members:	Tauseef Mushtaq, Mustafa Khan, Faryal Khan
Submission Date:	14 June 2026
Submission Deadline:	14 June 2026, 11:00 AM

THIS DOCUMENT IS CONFIDENTIAL AND INTENDED SOLELY FOR THE EVALUATION COMMITTEE OF DESC DIGITAL INNOVATION CENTER.

Letter of Transmittal

14 June 2026

The Director General

DESC Digital Innovation Center

Mardan, Khyber Pakhtunkhwa, Pakistan

Subject: Submission of Technical Proposal — Tender Reference DESC-MRD-2026-CNC-088

Dear Sir / Madam,

We, Team DeployX, are pleased to submit our Technical Proposal in response to the Request for Proposal (RFP) issued by DESC Digital Innovation Center, Mardan, for the Cloud-Native Application Orchestration and Deployment of the citizen portal.

We have carefully reviewed the RFP document and fully understand the scope, objectives, and technical requirements. Our team has already designed, developed, containerized, and deployed a fully functional MERN stack citizen portal using Kubernetes, demonstrating our practical readiness for this engagement.

This proposal presents our complete technical approach, toolstack, architecture, security strategy, monitoring plan, team composition, and project timeline. We are confident that our solution meets all requirements outlined in the RFP.

We confirm that:

- All information provided in this proposal is accurate and complete to the best of our knowledge.
- No pricing information has been included in this Envelope A (Technical Proposal).
- This proposal remains valid for a minimum period of ninety (90) days from the submission date.
- We are prepared to conduct a live deployment demonstration before the evaluation committee on the date specified by DESC.

Yours sincerely,

Nasrullah Khan

Team Leader — Team DeployX

Mardan, KPK, Pakistan

Table of Contents

Letter of Transmittal 2
1. Executive Summary 4
2. Project Reference 5
3. Scope and Objectives 6
4. Tools and Platforms in Scope 7
5. Team Profile and Roles & Responsibilities 8
6. RACI Chart 9
7. Proposed Technical Solution 10
8. System Architecture 12
9. Containerization Strategy 13
10. Kubernetes Cluster Design 14
11. CI/CD Pipeline 15
12. Infrastructure as Code — Terraform 16
13. Observability and Monitoring 17
14. Security Plan 18
15. Stateful Data Strategy 19
16. Planned AWS Cloud Architecture 20
17. Delivery and Reporting 21
18. Service Level Agreement (SLA) 22
19. Escalation Matrix 23
20. Bill of Materials 24
21. Risk Management 25
22. Demonstration Readiness 26
23. Project Timeline 27
24. RFP Compliance Matrix 28
25. Declaration and Signatures 29
Glossary of Terms 30

1. Executive Summary

Team DeployX is a group of four engineering students from Mardan, Khyber Pakhtunkhwa. This proposal is submitted in response to RFP DESC-MRD-2026-CNC-088, issued by DESC Digital Innovation Center for the modernization, orchestration, and managed operation of the DESC citizen portal.

The DESC citizen portal currently runs on old infrastructure that cannot handle busy periods, which causes the service to go down when citizens need it most. Our solution moves the portal to a container-based setup managed by Kubernetes, adds automated deployments, and sets up proper monitoring so the team always knows what is happening.

Our Solution at a Glance

Area	Technology	What it does
Containerization	Docker (multi-stage builds)	Packages the app so it runs the same in every environment
Orchestration	Kubernetes (Minikube / Cloud K8s)	Keeps the app running, restarts it on failure, scales it up automatically
CI/CD Automation	GitHub Actions	Code changes go live in under 5 minutes with no manual steps
Infrastructure as Code	Terraform	Entire infrastructure set up with one command
Monitoring	Prometheus + Grafana	Live graphs showing cluster and application health around the clock
Security	K8s Secrets, JWT, HTTPS	No credentials exposed anywhere in code or images

We have already built, containerized, and deployed a working MERN stack citizen portal on a Kubernetes cluster. Every tool and practice described in this proposal has been implemented and tested. This is not a theoretical approach — it is a working system ready for demonstration.

2. Project Reference

Field	Details
Tender Reference Number	DESC-MRD-2026-CNC-088
Issuing Organization	DESC Digital Innovation Center, Mardan
Project Title	Cloud-Native Application Orchestration and Deployment
Contract Type	Managed Service — Fixed Term
Contract Duration	12 Months (1 Year)
Submission Deadline	Sunday, 14 June 2026 at 11:00 AM (Sharp)
Submission Method	Two-Envelope System (Envelope A: Technical, Envelope B: Financial)
Proposal Validity	90 days from submission date
Bidder Name	Team DeployX
Team Leader	Nasrullah Khan
Contact Email	deployxteam@gmail.com
Contact Phone	+92-XXX-XXXXXXX
Address	Mardan, Khyber Pakhtunkhwa, Pakistan
GitHub Repository	github.com/Tauseef-Mushtaq/desc-portal
Docker Hub	hub.docker.com/u/tauseefmushtaq

3. Scope and Objectives

3.1 Project Scope

The scope of this engagement covers the complete modernization of the DESC citizen portal — from its current legacy infrastructure to a fully containerized, cloud-native platform. The selected team will be responsible for design, deployment, and 12-month managed operation of the new infrastructure.

In Scope

- Containerization of all application components using Docker
- Deployment and management of a Kubernetes cluster for orchestration
- Implementation of a CI/CD pipeline for automated build and deployment
- Infrastructure provisioning using Terraform (Infrastructure as Code)
- Integration of Prometheus and Grafana for monitoring and alerting
- Security hardening including secrets management and HTTPS configuration
- Persistent data management and backup strategy
- 12-month managed service including maintenance, updates, and support
- Live demonstration of all capabilities before the evaluation committee

Out of Scope

- Development of new features or modules not part of the existing citizen portal
- Hardware procurement or data center setup (cloud-based deployment assumed)
- End-user training for citizens using the portal
- Financial proposal details (covered separately in Envelope B)

3.2 Project Objectives

Objective	Description	Success Measure
High Availability	Portal is always accessible to citizens	99.99% uptime over 12 months
Scalability	Handle traffic spikes without manual work	HPA scales pods 2–5 automatically
Zero Downtime	Deploy updates without interrupting citizens	Rolling update strategy in K8s
Automation	Remove all manual deployment steps	CI/CD pipeline fully automated
Observability	Full visibility into system health	Grafana dashboards live 24/7
Security	No sensitive data exposed anywhere	All secrets in K8s Secret objects
Repeatability	Any team member can rebuild infrastructure	terraform apply recreates everything

4. Tools and Platforms in Scope

All tools below are open-source or freely available. Versions shown are what we have tested and confirmed working.

Category	Tool / Platform	Version	Purpose
Frontend Framework	React	18.x	Citizen-facing web interface
Backend Runtime	Node.js	20 LTS	Server-side JavaScript runtime
API Framework	Express.js	4.x	REST API development
Authentication	JSON Web Token (JWT)	9.x	Secure user authentication
File Uploads	Multer	1.x	Handling document uploads
Database	MongoDB	Latest	NoSQL document database
Web Server	Nginx	Alpine	Static file serving + API reverse proxy
Containerization	Docker	29.x	Application packaging into images
Container Compose	Docker Compose	3.8	Local multi-container orchestration
Orchestration	Kubernetes	v1.35.1	Production container management
Local K8s Cluster	Minikube	v1.38.1	Local Kubernetes environment
K8s CLI	kubectl	Latest	Command-line cluster management
K8s Package Manager	Helm	v3.21.0	Installing monitoring stack
CI/CD	GitHub Actions	Latest	Automated build and deployment pipeline
Container Registry	Docker Hub	Cloud	Storing and distributing Docker images
IaC Tool	Terraform	v1.15.5	Infrastructure as Code provisioning
Metrics Collection	Prometheus	v0.8.1	Cluster and application metrics
Dashboards	Grafana	Latest	Metrics visualisation and alerting
Alerting	AlertManager	Latest	Alert routing and notification
OS / Environment	Ubuntu 22.04 (WSL2)	LTS	Development and deployment OS
Version Control	Git + GitHub	Latest	Source code and config management

5. Team Profile and Roles & Responsibilities

5.1 About Team DeployX

Team DeployX is a group of four engineering students from Mardan, Khyber Pakhtunkhwa. Everyone on the team has hands-on experience in their area, and we have already completed a working implementation of the DESC citizen portal as a proof of concept before submitting this proposal.

5.2 Team Members

Name	Role	Qualification	Key Skills
Nasrullah Khan	Team Leader & Cloud Architect	BS Computer Science	Kubernetes, cloud architecture, project management, technical documentation
Tauseef Mushtaq	DevOps Engineer	BS Software Engineering	Docker, CI/CD, Terraform, GitHub Actions, Linux, shell scripting
Mustafa Khan	Backend Developer	BS Computer Science	Node.js, Express, MongoDB, REST APIs, JWT, Mongoose
Faryal Khan	Frontend Developer	BS Information Technology	React 18, JavaScript, Nginx, CSS, responsive UI design

5.3 Roles and Responsibilities

Name	Responsibilities
Nasrullah Khan	Overall project leadership and delivery. Architecture design and review. Stakeholder communication with DESC. Technical decision-making. Final approval of all deliverables. Monthly reporting to DESC management.
Tauseef Mushtaq	CI/CD pipeline design and maintenance. Docker image building and pushing to registry. Terraform infrastructure management. Kubernetes cluster operations. Monitoring stack setup and alerting. Security hardening and secrets management.
Mustafa Khan	Backend API development and maintenance. MongoDB database management. Authentication and authorisation logic. API documentation. Performance work on backend services. Database backup and recovery.
Faryal Khan	Frontend React development. Nginx configuration and reverse proxy setup. UI improvements and bug fixes. Cross-browser testing. Static asset work. Responsive design maintenance.

6. RACI Chart

The RACI chart shows who is Responsible, Accountable, Consulted, and Informed for each major activity. The three supervisors above our team are listed in the first columns — they are consulted and kept informed but the team itself owns delivery.

R = Responsible **A** = Accountable **C** = Consulted **I** = Informed

Activity	Sir Rafid Imran (TL Supervisor)	Nasrullah (Team Lead)	Tauseef (DevOps)	Mustafa (Backend)	Faryal (Frontend)
Project planning & scheduling	C	A	R	C	C
Architecture design	C	A/R	C	C	C
Docker containerization	I	A	R	C	R
Kubernetes cluster setup	C	A	R	I	I
CI/CD pipeline implementation	I	A	R	I	I
Terraform IaC configuration	I	A	R	I	I
Backend API development	I	A	C	R	I
Frontend development	I	A	C	I	R
MongoDB database management	I	A	I	R	I
Nginx configuration	I	A	C	I	R
Prometheus & Grafana setup	I	A	R	I	I
Security implementation	C	A	R	C	C
Live demo preparation	C	A	R	C	C
Monthly reporting to DESC	A	R	C	C	C
Incident response	I	A	R	C	C
Documentation	I	A	R	R	R

7. Proposed Technical Solution

7.1 Solution Overview

Our solution moves the DESC citizen portal from a single legacy server to a set of separate, independently running services — each in its own Docker container and managed by Kubernetes. The three services are frontend, backend, and database.

This means a problem with one service does not bring down the others. Each service can be scaled on its own depending on demand, and updates can be pushed without taking the portal offline.

7.2 Application Stack

Layer	Technology	Port	Description
Presentation	React 18 + Nginx	80 / 3000	The citizen-facing web page. React handles the UI. Nginx serves the built files and forwards API calls to the backend.
Application	Node.js + Express + JWT	5000	REST API server handling business logic, user login, file uploads, and data operations.
Data	MongoDB	27017	Stores all citizen requests, user accounts, and service records on a Kubernetes PersistentVolumeClaim.

7.3 How the Services Work Together

When a citizen opens the portal:

- Browser sends request to http://desc-portal.local
- Nginx Ingress receives it and routes to the Frontend service
- Frontend pod serves the React page to the browser
- When the citizen submits a form, browser calls /api/...
- Nginx inside the frontend container proxies /api/ to the Backend service
- Backend validates the JWT token, runs the business logic, and queries MongoDB
- MongoDB returns data to the backend, which sends a JSON response back
- Frontend displays the result to the citizen

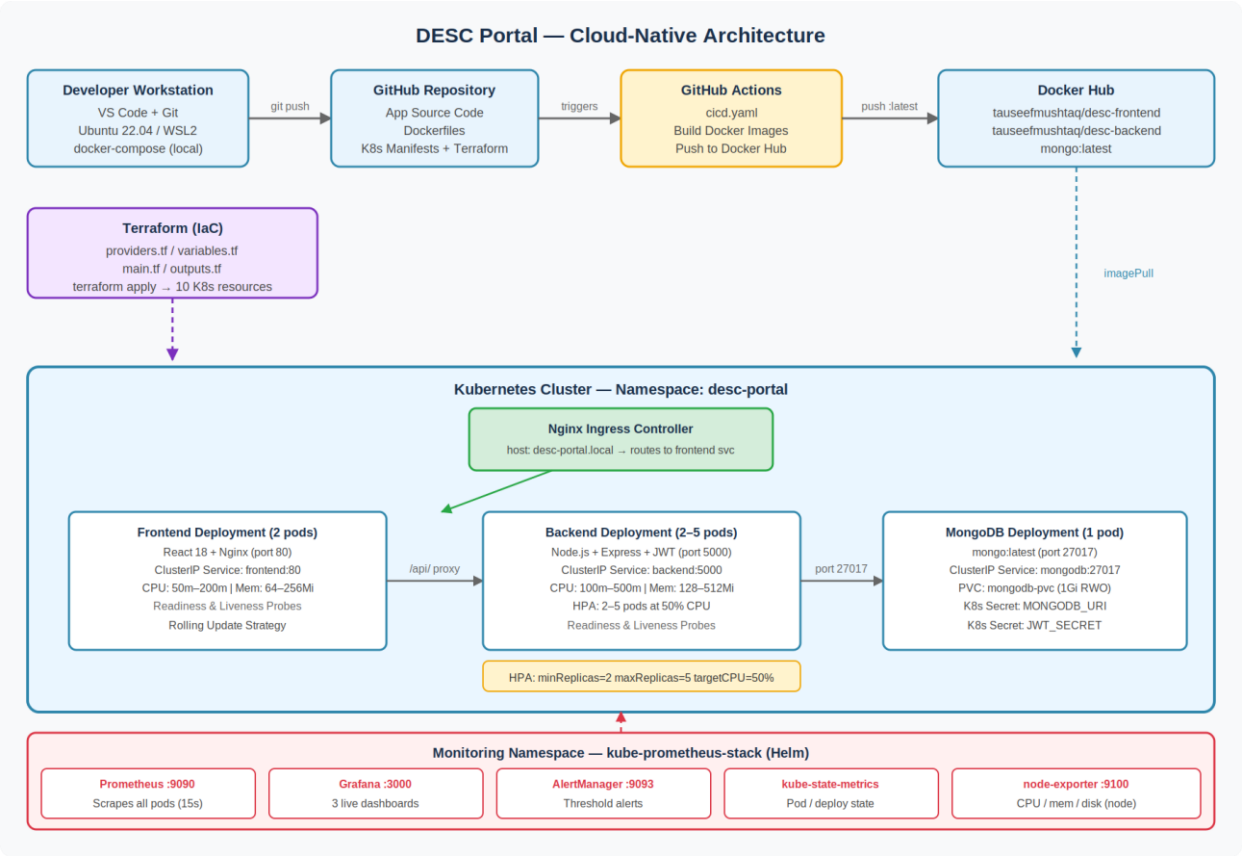
7.4 Why This Architecture Works for DESC

Challenge	How Our Architecture Solves It
Load spikes crash the portal	HPA adds more backend pods when CPU exceeds 50%, handling the extra load automatically
Updates cause downtime	Kubernetes rolling updates replace old pods one at a time — portal stays live throughout
A server crash takes everything down	2 replicas each of frontend and backend — if one pod fails, the other keeps serving citizens
Hard to reproduce on a new server	Terraform creates the entire infrastructure with one command on any machine
No visibility when things go wrong	Prometheus collects metrics every 15 seconds and Grafana shows live dashboards

8. System Architecture

The diagram below shows the complete architecture: the developer workflow on the left, the CI/CD pipeline in the middle, the Kubernetes cluster, and the monitoring stack at the bottom.

Figure 1 — Current Deployment Architecture (Kubernetes on Minikube)



8.1 Architecture Layers

Layer	Components	Role
Developer Workstation	VS Code, Git, Ubuntu (WSL2)	Where code is written and committed
Source Control	GitHub repository	Stores all code, Dockerfiles, K8s manifests, and Terraform configs
CI/CD Layer	GitHub Actions (cicd.yaml)	Builds and publishes Docker images automatically on every push to main
Container Registry	Docker Hub	Stores versioned Docker images accessible by Kubernetes
IaC Layer	Terraform	Provisions all Kubernetes resources automatically
Orchestration Layer	Kubernetes (desc-portal namespace)	Runs, scales, and heals all application pods
Networking Layer	Nginx Ingress + K8s Services	Routes traffic correctly between browser and services
Storage Layer	PersistentVolumeClaim	Keeps MongoDB data safe across pod restarts
Secrets Layer	Kubernetes Secrets	Stores credentials securely, injected at runtime
Monitoring Layer	Prometheus + Grafana (monitoring namespace)	Collects and displays all cluster metrics

9. Containerization Strategy

9.1 Docker Image Design

Each service is built into a Docker image using a multi-stage build. Multi-stage builds keep the final image small by throwing away build tools and temporary files before creating the production image.

Service	Base Image	Build Stages	Final Size
Frontend	node:20-alpine + nginx:alpine	Stage 1: Node builds the React app. Stage 2: Nginx serves the /build folder.	~25 MB
Backend	node:20-alpine	Single stage: copies source, installs production dependencies only.	~180 MB
MongoDB	mongo:latest	No custom build — official image used directly.	~700 MB

9.2 Nginx Reverse Proxy

The frontend Nginx config serves two purposes:

- Serves the React app for all routes so React Router works correctly on page refresh
- Proxies all /api/ and /uploads/ requests to the backend service — this removes CORS issues completely

9.3 Docker Security Practices

- A .dockerignore file stops .env, node_modules, and other sensitive files from being copied into images
- The backend CMD uses npm start (production mode), not nodemon (development mode)
- No environment variables or secrets are hardcoded in any Dockerfile
- Bind mounts are not used in production — code is baked into the image at build time
- Images are tagged :latest for CI/CD and stored on Docker Hub under a secure account

10. Kubernetes Cluster Design

10.1 Namespace and Resource Organisation

All application resources are deployed in a dedicated namespace called desc-portal. This isolates the portal from other workloads and keeps resource management, access control, and monitoring straightforward.

10.2 Kubernetes Resources

Resource	Name	Configuration	Purpose
Namespace	desc-portal	—	Isolated workspace for all portal resources
Deployment	frontend	2 replicas, rolling update, cpu: 50m–200m, mem: 64–256Mi	Runs React+Nginx pods
Deployment	backend	2 replicas, HPA-managed, cpu: 100m–500m, mem: 128–512Mi	Runs Node.js API pods
Deployment	mongodb	1 replica	Runs MongoDB pod
Service	frontend	ClusterIP, port 80	Internal address for frontend pods
Service	backend	ClusterIP, port 5000	Internal address for backend pods
Service	mongodb	ClusterIP, port 27017	Internal address for MongoDB
Ingress	desc-portal-ingress	Nginx, host: desc-portal.local	External entry point, routes to frontend
HPA	backend-hpa	min: 2, max: 5 replicas, CPU target: 50%	Auto-scales backend based on CPU load
PVC	mongodb-pvc	1Gi, ReadWriteOnce	Persistent storage for MongoDB data
Secret	backend-secret	Opaque	MONGODB_URI, JWT_SECRET, PORT, NODE_ENV
ServiceMonitor	desc-portal-monitor	namespace: desc-portal	Tells Prometheus which pods to scrape

10.3 Health Probes

Both backend pods have Kubernetes health probes so traffic only goes to healthy pods:

- Readiness Probe — checks /api/health every 5 seconds with a 10-second initial delay. Pod only receives traffic when it passes.
- Liveness Probe — checks /api/health every 10 seconds with a 15-second initial delay. Pod is restarted if it stops responding.

11. CI/CD Pipeline

11.1 Pipeline Overview

Our CI/CD pipeline automates the whole delivery process. A developer pushes code to GitHub, and within a few minutes a new version of the application is built and available — no manual steps needed.

11.2 Pipeline Flow

Stage	Tool	Action	Trigger
Source	Git + GitHub	Developer pushes code to main branch	git push origin main
Build	GitHub Actions	Workflow file (cicd.yaml) is triggered	Push to main branch
Containerize	docker/build-push-action@v6	Builds new Docker image for frontend and backend	Automatic
Publish	Docker Hub	Pushes updated images tagged :latest	Automatic
Deploy	kubectl rollout restart	Kubernetes pulls new image and restarts pods	Manual trigger after CI/CD

11.3 GitHub Actions Workflow Details

- Workflow file: .github/workflows/cicd.yaml
- Trigger: on push to main branch
- Actions used: actions/checkout@v4, docker/login-action@v3, docker/build-push-action@v6
- Credentials stored securely as GitHub repository secrets (DOCKERHUB_USERNAME, DOCKERHUB_TOKEN)
- Frontend and backend images are built and pushed in parallel for faster delivery
- All actions use Node.js 24 compatible versions — no deprecation warnings

11.4 Zero-Downtime Deployment

Kubernetes rolling update strategy means the portal stays up during every deployment. New pods start first, pass health checks, then the old ones are stopped. At no point are all pods offline at the same time.

12. Infrastructure as Code — Terraform

12.1 Why Terraform

Without Terraform, setting up the Kubernetes infrastructure means running more than ten kubectl commands in the right order. One mistake can cause failures. Terraform solves this by writing the desired state in code and applying it automatically, correctly, every time.

12.2 Terraform Files

File	Purpose	Key Content
providers.tf	Connects Terraform to the cluster	hashicorp/kubernetes provider v2.23.0, reads ~/.kube/config
variables.tf	Defines all configurable parameters	Image names, replica counts, JWT secret, MongoDB URI, storage size
main.tf	Creates all Kubernetes resources	Namespace, secret, PVC, 3 deployments, 3 services, HPA
outputs.tf	Shows results after apply	Summary of all created resources and their values

12.3 Resources Provisioned

Running terraform apply creates these 10 resources automatically:

- kubernetes_namespace — creates the desc-portal isolated workspace
- kubernetes_secret — stores all environment variables securely
- kubernetes_persistent_volume_claim — reserves 1Gi storage for MongoDB
- kubernetes_deployment (MongoDB) — runs the database pod with PVC mount
- kubernetes_deployment (Backend) — runs 2 API server pods with health probes
- kubernetes_deployment (Frontend) — runs 2 React/Nginx pods
- kubernetes_service (MongoDB, Backend, Frontend) — creates internal service addresses
- kubernetes_horizontal_pod_autoscaler_v2 — configures auto-scaling for backend

12.4 Deployment Summary Output

After terraform apply finishes, it prints a summary like this:

```
=== DESC Portal Deployment Summary ===
Namespace  : desc-portal
Backend    : tauseefmushtaq/desc-backend:latest (2 replicas)
Frontend   : tauseefmushtaq/desc-frontend:latest (2 replicas)
MongoDB    : mongo:latest (1 replica + 1Gi PVC)
HPA        : 2-5 replicas at 50% CPU threshold
```

13. Observability and Monitoring

13.1 Monitoring Stack

We installed the kube-prometheus-stack using Helm. This deploys Prometheus, Grafana, AlertManager, kube-state-metrics, and node-exporter in a dedicated monitoring namespace in one command.

Component	Role	Port	Notes
Prometheus	Collects and stores metrics from all pods every 15 seconds	9090	
Grafana	Displays metrics as live graphs and dashboards	3000	3 dashboards configured
AlertManager	Routes alerts when thresholds are breached	9093	
kube-state-metrics	Exposes K8s object state — pods, deployments, services	8080	
node-exporter	Exposes host-level metrics — CPU, memory, disk, network	9100	
ServiceMonitor	Tells Prometheus to scrape the desc-portal namespace	N/A	Custom CRD

13.2 Grafana Dashboards

Dashboard	Namespace	What It Shows
Kubernetes / Compute Resources / Pod	desc-portal	CPU and memory per pod, throttling, quota
Kubernetes / Compute Resources / Namespace (Pods)	desc-portal	Overview of all pods with resource usage
Node Exporter / Use Method / Cluster	monitoring	Cluster node health — CPU, memory, disk, network

13.3 Key Metrics Monitored

- CPU utilisation per pod — alert if any pod consistently exceeds 80% of its limit
- Memory usage per pod — alert if memory approaches the configured limit
- Pod restart count — alert if any pod restarts more than 3 times in 10 minutes
- HPA replica count — tracks when auto-scaling events happen
- Node CPU and memory — overall health of the cluster node

14. Security Plan

14.1 Security Principles

Security is applied at every layer — from the code repo to the running containers. The main rule is: no credential should ever appear in any log, code file, or Docker image layer.

Layer	Security Measure	How We Did It
Source Code	No secrets in code	All credentials loaded from environment variables. .env in .gitignore
Docker Images	No secrets in images	.dockerignore stops .env files from being copied into images
Container Registry	Secure credentials	Docker Hub access token (not password) stored as GitHub Secret
Kubernetes Secrets	Encrypted credential storage	MONGODB_URI, JWT_SECRET stored as K8s Opaque Secret, injected at runtime
Authentication	JWT-based API security	All API endpoints require a valid JWT token except login and register
Namespace Isolation	Resource separation	All resources in desc-portal namespace, isolated from other workloads
Network	Internal-only services	All services use ClusterIP — only the Ingress is externally accessible
HTTPS	TLS termination	Nginx Ingress configured for TLS in production using SSL certificates
Image Pull Policy	Always use tested image	imagePullPolicy: Always ensures only tested images run in production

14.2 Secret Management Flow

- Developer stores sensitive values in .env file locally — never committed to Git
- In production, all secrets are created as Kubernetes Secret objects
- Pods receive secrets as environment variables injected by Kubernetes at startup
- No secret value ever appears in any log, console output, or Docker image layer

15. Stateful Data Strategy

15.1 Data Persistence

MongoDB stores data on disk. In Kubernetes, pods are temporary by design — if a pod is deleted and recreated, its local storage disappears. We solve this with a PersistentVolumeClaim (PVC).

Property	Value
PVC Name	mongodb-pvc
Storage Size	1 Gigabyte (expandable)
Access Mode	ReadWriteOnce — one pod reads and writes at a time
Storage Class	standard (Minikube default provisioner)
Mount Path	/data/db inside the MongoDB container
Lifecycle	Independent of the pod — data survives restarts and updates

15.2 Backup Strategy

Environment	Database	Backup Method	Frequency
Demo / Local	MongoDB container + PVC	PVC data survives restarts automatically	N/A
Production (recommended)	MongoDB Atlas	Automated cloud backups managed by Atlas	Daily (configurable)
Production (alternative)	Self-hosted MongoDB	mongodump to object storage (S3 or similar)	Daily via cron job

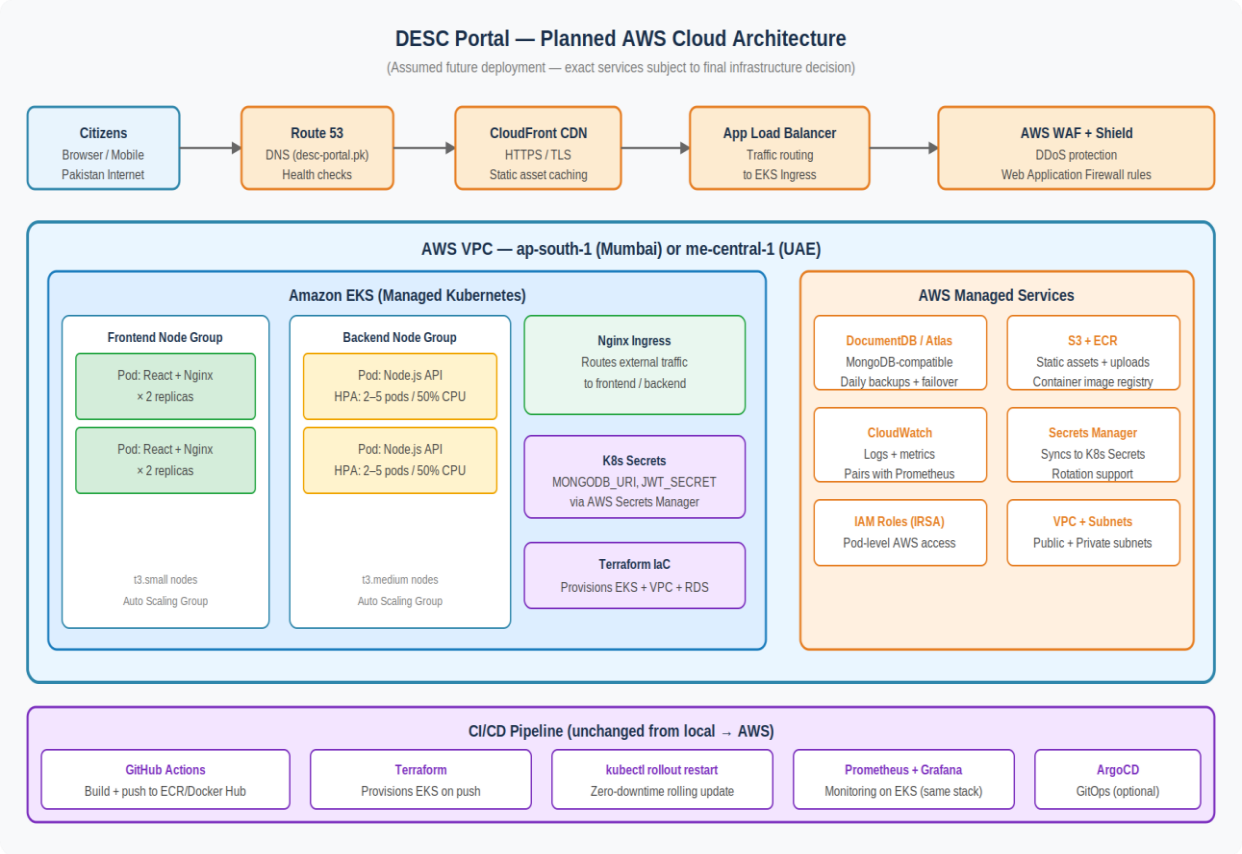
15.3 MongoDB Atlas Migration

Switching from the local MongoDB container to Atlas in production requires only one change: updating the MONGODB_URI in the Kubernetes Secret. The .env.example file in our repo already has a commented Atlas connection string template. No code changes are needed.

16. Planned AWS Cloud Architecture

Note: This section describes the assumed future production deployment on AWS. The exact services and configuration are subject to final infrastructure decisions and budget approved in Envelope B. The architecture below is based on standard AWS best practices for hosting a containerized MERN stack application.

Figure 2 — Planned AWS Production Architecture



16.1 Why AWS

AWS is the most widely used cloud platform and is well-supported in Pakistan. The region closest to Mardan is ap-south-1 (Mumbai, India) or me-central-1 (UAE). Either gives good latency for users in KPK.

AWS Service	Replaces / Maps To	Why
Amazon EKS	Minikube (local Kubernetes)	Managed Kubernetes — no need to maintain the control plane manually
Amazon ECR	Docker Hub	Private container registry inside AWS, faster image pulls for EKS
Amazon DocumentDB / Atlas	Local MongoDB + PVC	Managed MongoDB-compatible database with automated backups and failover
Amazon S3	Local file storage	Stores uploaded files and static assets reliably
AWS ALB (Load Balancer)	Nginx Ingress (local)	Distributes traffic across EKS nodes and handles SSL termination
Amazon CloudFront	Direct browser access	CDN to cache static assets closer to users across Pakistan
Route 53	Local /etc/hosts	DNS management for desc-portal.pk or similar domain
AWS Secrets Manager	Kubernetes Secrets (local)	Manages and rotates secrets, syncs to K8s Secrets via IRSA

AWS Service	Replaces / Maps To	Why
AWS CloudWatch	Grafana (local port-forward)	Centralised logging, can work alongside Prometheus and Grafana on EKS
AWS WAF + Shield	Not present locally	Protects against DDoS and common web attacks
IAM Roles (IRSA)	Local kubeconfig	Lets pods access AWS services securely without hardcoded credentials
VPC + Subnets	Single network locally	Public subnet for ALB, private subnets for EKS nodes and database

16.2 What Stays the Same

The move to AWS does not change the application or most of the tooling. The same Docker images, the same Kubernetes manifests, and the same Terraform scripts are used. The main differences are:

- terraform apply points to EKS instead of Minikube
- Docker images are pushed to ECR instead of Docker Hub (or both)
- MongoDB connection string points to Atlas or DocumentDB instead of the local container
- Nginx Ingress is replaced by AWS ALB Ingress Controller
- The GitHub Actions pipeline adds an AWS credentials step for ECR push

16.3 Estimated AWS Services Needed

Detailed costing is in Envelope B. The main AWS services expected are:

- EKS cluster — 2–3 worker nodes (t3.small for frontend, t3.medium for backend)
- DocumentDB or MongoDB Atlas cluster — M10 tier minimum for production
- ALB — one load balancer for all traffic
- S3 bucket — for file uploads and static assets
- Route 53 hosted zone — for DNS management
- CloudFront distribution — optional, recommended for better performance

17. Delivery and Reporting

17.1 Delivery Plan

Phase	Duration	Deliverables	Acceptance Criteria
Phase 1 — Setup	Weeks 1–2	Cloud infrastructure provisioned via Terraform. K8s cluster live. CI/CD pipeline operational.	terraform apply succeeds. kubectl get nodes returns Ready. Pipeline triggers on push.
Phase 2 — Migration	Weeks 3–4	Legacy app containerized. K8s manifests deployed. Data migrated.	All 5 pods Running. App accessible via Ingress. Data verified in MongoDB.
Phase 3 — Automation	Weeks 5–6	GitHub Actions fully automated. Rolling updates tested. Zero-downtime deployment verified.	Code push triggers pipeline. Zero-downtime update demonstrated.
Phase 4 — Monitoring	Weeks 7–8	Prometheus and Grafana deployed. Dashboards configured. Alerting rules set.	Grafana dashboards live. Alert fires on test threshold breach.
Phase 5 — Hardening	Weeks 9–12	Security review. Load testing. Documentation. DESC team training.	Load test passes. Security scan clean. Documentation approved by DESC.
Phase 6 — Operations	Months 3–12	Ongoing managed service. Incident response. Monthly reports. Quarterly reviews.	SLA targets met monthly. Reports submitted on time.

17.2 Reporting Schedule

Report	Frequency	Contents	Submitted To
Weekly Status Report	Every Monday	Tasks done, tasks planned, blockers, pod health summary	Project coordinator at DESC
Monthly Operations Report	Monthly	Uptime %, incidents, deployments, resource usage, Grafana screenshots	DESC management
Quarterly Review Report	Quarterly	SLA performance, cost analysis, recommendations, next-quarter roadmap	DESC Director General
Incident Report	Within 24 hours of incident	Incident description, root cause, resolution, preventive steps	DESC technical team

18. Service Level Agreement (SLA)

18.1 Service Availability

Service	Target Availability	Measurement Period	Calculation Method
DESC Citizen Portal	99.99%	Monthly	(Total minutes − Downtime minutes) / Total minutes × 100
Kubernetes Cluster	99.95%	Monthly	Measured via Prometheus uptime metrics
CI/CD Pipeline	99.90%	Monthly	GitHub Actions + Docker Hub availability
Monitoring Dashboards	99.90%	Monthly	Grafana and Prometheus pod uptime

18.2 Response and Resolution Times

Priority	Description	Response Time	Resolution Time	
P1 — Critical	Portal completely inaccessible to all citizens	15 minutes	4 hours	
P2 — High	Major feature unavailable or significant slowdown	30 minutes	8 hours	
P3 — Medium	Minor feature down or partial issues affecting some users	2 hours	24 hours	
P4 — Low	Cosmetic issues, minor bugs, non-urgent requests	8 hours	72 hours	

18.3 SLA Penalties

Monthly Uptime Achieved	Penalty
99.99% and above	No penalty — full payment as agreed
99.90% to 99.98%	5% reduction of monthly retainer fee
99.00% to 99.89%	10% reduction of monthly retainer fee
Below 99.00%	15% reduction + formal incident review meeting with DESC

18.4 Exclusions

- Scheduled maintenance windows communicated at least 48 hours in advance
- Force majeure events — natural disasters, power outages, or ISP failures beyond our control
- Downtime caused by changes made by DESC staff without coordination with Team DeployX
- Third-party service outages (Docker Hub, GitHub) beyond 30 minutes

19. Escalation Matrix

19.1 Escalation Levels

Level	When to Escalate	Contact (DeployX)	Contact (DESC)	Expected Action
Level 1	P3 or P4 not resolved within SLA	Faryal Khan / Mustafa Khan	DESC Help Desk	Engineer resolves within SLA time
Level 2	P2 or Level 1 unresolved after 4 hours	Tauseef Mushtaq (DevOps)	DESC IT Manager	Senior engineer takes over. Root cause analysis starts.
Level 3	P1 or Level 2 unresolved after 2 hours	Nasrullah Khan (Team Lead)	DESC Director General	Full team mobilised. Emergency response. Executive communication.
Level 4	Repeated SLA breaches or contractual dispute	Nasrullah Khan	DESC Legal / Procurement	Formal review meeting. Contract amendment if needed.

19.2 Escalation Process

- Step 1: Issue detected via Prometheus alert or reported by DESC staff
- Step 2: On-duty engineer acknowledges within response time defined in SLA
- Step 3: If not resolved within 50% of resolution time, escalate to next level
- Step 4: All P1 and P2 issues documented in an Incident Report within 24 hours
- Step 5: Post-incident review within 48 hours for all P1 incidents

20. Bill of Materials

Note: All tools listed below are open-source or free to use. Cloud infrastructure costs are covered in the Financial Proposal (Envelope B).

#	Item	Category	Version / Specification	License
1	React	Frontend Framework	18.x	MIT
2	Node.js	Backend Runtime	20 LTS	MIT
3	Express.js	API Framework	4.x	MIT
4	MongoDB	Database	Latest	SSPL
5	Nginx	Web Server	Alpine	BSD-2
6	Docker Engine	Containerization	29.x	Apache 2.0
7	Docker Hub	Container Registry	Cloud	Proprietary (Free Tier)
8	Kubernetes	Orchestration	v1.35.1	Apache 2.0
9	Minikube	Local K8s	v1.38.1	Apache 2.0
10	kubectl	K8s CLI	Latest	Apache 2.0
11	Helm	K8s Package Manager	v3.21.0	Apache 2.0
12	GitHub Actions	CI/CD	Latest	Proprietary (Free Tier)
13	Terraform	IaC Tool	v1.15.5	BSL
14	Prometheus	Metrics Collection	v0.8.1	Apache 2.0
15	Grafana	Dashboards	Latest	AGPL v3
16	AlertManager	Alert Routing	Latest	Apache 2.0
17	kube-state-metrics	K8s State Exporter	Latest	Apache 2.0
18	node-exporter	Node Metrics	Latest	Apache 2.0
19	Git	Version Control	Latest	GPL v2
20	Ubuntu 22.04 LTS	Operating System	22.04	GPL
21	WSL2	Windows Linux Layer	Latest	Proprietary (Free)
22	JSON Web Token (JWT)	Auth Library	9.x	MIT
23	Multer	File Upload Middleware	1.x	MIT
24	Mongoose	MongoDB ODM	7.x	MIT
25	kube-prometheus-stack	Helm Chart (monitoring)	Latest	Apache 2.0

All cloud infrastructure costs (compute, storage, networking) are listed in the Financial Proposal submitted separately in Envelope B.

21. Risk Management

Risk	Likelihood	Impact	Mitigation
Pod failure causing service disruption	Low	High	2 replicas of frontend and backend. K8s self-heals automatically within seconds.
Database data loss	Very Low	Critical	PVC keeps data across restarts. Production uses MongoDB Atlas with daily backups.
CI/CD pipeline failure	Low	Medium	Pipeline failures don't affect the running app — previous image keeps running.
Network connectivity issues	Medium	Medium	Retry logic in app. K8s readiness probes stop unhealthy pods from getting traffic.
Credential exposure / security breach	Very Low	Critical	All secrets in K8s Secrets. No credentials in code or images. JWT protects all APIs.
Traffic spike beyond HPA maximum	Low	Medium	HPA handles up to 5 pods. Cloud provider can add nodes if needed.
Team member unavailability	Low	Medium	RACI chart + cross-training. All procedures documented. Any member can cover any task.
Docker Hub downtime	Very Low	Low	Images pre-loaded into Minikube using imagePullPolicy: IfNotPresent.
Third-party library vulnerability	Medium	Medium	npm audit runs in CI/CD pipeline. Dependencies updated regularly.
Scope creep	Medium	Medium	Scope defined in Section 3. Changes require written approval from DESC management.

22. Demonstration Readiness

22.1 Demo Overview

Team DeployX is ready to conduct a live deployment demonstration before the DESC evaluation committee on the date specified by DESC, following approval of this proposal. The demonstration will show the complete platform in action — the running application, self-healing, auto-scaling, CI/CD pipeline, Terraform provisioning, and live monitoring dashboards.

22.2 Demo Script

Step	Duration	Action	Tool / Command	What It Proves
1	2 min	Start cluster, show node status	minikube start --driver=docker + kubectl get nodes	Kubernetes environment is operational
2	2 min	Show all 5 pods running	kubectl get all -n desc-portal	All services are healthy
3	3 min	Open portal, login as admin and citizen	Browser at minikube service URL	Application is fully functional
4	2 min	Show GitHub Actions green pipeline	GitHub Actions tab in browser	CI/CD automation works end to end
5	3 min	Delete a backend pod, watch restart	kubectl delete pod <name>	K8s self-healing works in real time (~10 sec)
6	2 min	Show HPA status	kubectl get hpa -n desc-portal	Auto-scaler is configured and watching CPU
7	2 min	Show live resource usage	kubectl top pods	CPU and memory monitoring is working
8	3 min	Open Grafana dashboards	Grafana at localhost:3000	Prometheus and Grafana monitoring is live
9	3 min	Run terraform apply	terraform apply (in terminal)	10 K8s resources created from one command
10	2 min	Show secrets management	kubectl get secret + explain flow	Security implementation is in place
11	2 min	Questions from committee	Open Q&A	Team readiness and understanding

22.3 Technical Requirements for Demo

- Laptop with Docker Desktop running and WSL2 enabled
- Active internet connection (for Docker Hub image pulling if needed)
- Projector or screen for live display
- Minikube cluster pre-started with all pods in Running state
- Grafana port-forward running on localhost:3000
- Demo data pre-seeded in MongoDB using the seed script

23. Project Timeline

Phase	Timeline	Key Activities	Milestone
Phase 1 — Infrastructure Setup	Month 1, Week 1–2	Cloud provider account setup. Terraform configured. K8s cluster provisioned. CI/CD pipeline deployed and tested.	K8s cluster live. Pipeline green.
Phase 2 — Application Migration	Month 1, Week 3–4	Legacy app containerized. Docker images built and pushed. K8s manifests deployed. Initial data migration done.	All pods Running. App accessible.
Phase 3 — Automation	Month 2, Week 5–6	CI/CD fully automated. Rolling updates tested. Zero-downtime deployment verified. GitOps workflow set up.	Zero-downtime deployment demonstrated.
Phase 4 — Monitoring	Month 2, Week 7–8	Prometheus deployed. Grafana dashboards configured. Alerting rules set. Team notified of alert channels.	Live dashboards operational.
Phase 5 — Hardening & Training	Month 2–3, Week 9–12	Security review done. Load testing performed. DESC team trained. Documentation handed over.	Security scan clean. Team trained.
Phase 6 — Managed Operations	Month 3–12	Ongoing monitoring, maintenance, and support. Monthly reports. Quarterly reviews. Incident response.	Monthly SLA reports submitted.

24. RFP Compliance Matrix

The table below confirms our compliance with every requirement in RFP DESC-MRD-2026-CNC-088.

RFP Requirement	RFP Ref	Our Solution	Status
Containerize monolithic components into microservices	RFP §2	Docker multi-stage builds for frontend, backend, and MongoDB	COMPLIANT
Design and provision Kubernetes cluster	RFP §2	Minikube K8s with namespace, deployments, services, ingress	COMPLIANT
Automated scaling	RFP §2	HPA auto-scales backend pods 2–5 at 50% CPU threshold	COMPLIANT
Self-healing pods	RFP §2	K8s restarts failed pods automatically via deployment controller	COMPLIANT
Load balancing	RFP §2	Nginx Ingress + Kubernetes ClusterIP services	COMPLIANT
Implement CI/CD pipeline	RFP §2	GitHub Actions with checkout@v4 and build-push-action@v6	COMPLIANT
Automated testing and zero-downtime rollouts	RFP §2	Rolling update strategy configured in all deployments	COMPLIANT
Infrastructure as Code (Terraform or Ansible)	RFP §2	Terraform v1.15.5 with hashicorp/kubernetes provider	COMPLIANT
Integrate Prometheus for metrics	RFP §2	kube-prometheus-stack Helm chart in monitoring namespace	COMPLIANT
Integrate Grafana for dashboards	RFP §2	Three dashboards configured with live cluster and pod metrics	COMPLIANT
Track cluster health and resource utilisation	RFP §2	ServiceMonitor, node-exporter, kube-state-metrics all deployed	COMPLIANT
Kubernetes architecture diagram (Envelope A)	RFP §3	Architecture diagram included in Section 8	COMPLIANT
GitOps/deployment workflow (Envelope A)	RFP §3	CI/CD workflow described in Section 11 with cicd.yaml details	COMPLIANT
Chosen toolstack (Envelope A)	RFP §3	Complete toolstack with versions in Section 4	COMPLIANT
Stateful data strategy (Envelope A)	RFP §3	PVC for MongoDB, Atlas migration path in Section 15	COMPLIANT
Security strategy (Envelope A)	RFP §3	K8s Secrets, JWT, HTTPS, .dockerignore in Section 14	COMPLIANT
Live deployment demonstration readiness	RFP §4	Full demo script in Section 22. System ready for demo.	COMPLIANT
No pricing in Envelope A	RFP §3	No financial information in this document	COMPLIANT

25. Declaration and Signatures

We, the undersigned members of Team DeployX, hereby declare that:

- All information, statements, and representations in this Technical Proposal are true, accurate, and complete to the best of our knowledge.
- We have not and will not engage in any collusive, fraudulent, or corrupt practices in connection with this tender.
- We have read, understood, and accepted all terms and conditions in RFP DESC-MRD-2026-CNC-088.
- This proposal contains no pricing information, in line with the two-envelope submission requirement.
- This proposal remains valid for ninety (90) calendar days from the date of submission.
- We are prepared to sign a formal contract and start work within fourteen (14) days of receiving a Letter of Intent from DESC.
- All team members named in this proposal are available and committed for the full duration of the contract.

Name	Role	Signature	Date	
Nasrullah Khan	Team Leader & Cloud Architect	_____	14 June 2026	
Tauseef Mushtaq	DevOps Engineer	_____	14 June 2026	
Mustafa Khan	Backend Developer	_____	14 June 2026	
Faryal Khan	Frontend Developer	_____	14 June 2026	

Official Stamp / Seal of Team DeployX

Glossary of Terms

Term	Definition
CI/CD	Continuous Integration / Continuous Deployment — automated process of building, testing, and deploying software changes
Container	A lightweight, standalone package that includes everything needed to run a piece of software
Docker	A tool for creating, running, and managing containers
Docker Hub	An online registry where Docker images are stored and shared
Dockerfile	A text file with instructions to build a Docker image
Grafana	An open-source tool for visualising metrics as charts and dashboards
Helm	A package manager for Kubernetes that simplifies deployment of complex applications
HPA	Horizontal Pod Autoscaler — automatically scales the number of pods based on CPU or memory usage
IaC	Infrastructure as Code — managing infrastructure using code rather than manual processes
Ingress	A Kubernetes resource that manages external access to services inside the cluster
JWT	JSON Web Token — a compact way to securely transmit information, used for authentication
K8s	Common abbreviation for Kubernetes (K + 8 letters + s)
kubectl	Command-line tool for interacting with a Kubernetes cluster
Kubernetes	An open-source system for automating deployment, scaling, and management of containerized applications
MERN	MongoDB, Express.js, React, Node.js — a popular web development technology stack
Microservices	An architectural style where an application is built as small, independently deployable services
Minikube	A tool that runs a single-node Kubernetes cluster locally on a personal computer
MongoDB	A NoSQL document database that stores data in flexible JSON-like documents
Namespace	A Kubernetes feature for dividing cluster resources between multiple users or applications
Nginx	A web server used as a reverse proxy, load balancer, and static file server
Node.js	A JavaScript runtime for building server-side applications
PVC	PersistentVolumeClaim — a request for storage in Kubernetes that persists beyond the life of a pod
Pod	The smallest deployable unit in Kubernetes, containing one or more containers
Prometheus	An open-source monitoring toolkit that collects metrics from targets at regular intervals
RACI	Responsible, Accountable, Consulted, Informed — a matrix clarifying roles in project activities
React	A JavaScript library for building user interfaces
Replica	A copy of a pod running at the same time to provide redundancy and distribute load
Rolling Update	A Kubernetes strategy that updates pods gradually to avoid downtime
SLA	Service Level Agreement — a commitment between provider and client defining expected service levels
Terraform	An IaC tool by HashiCorp that provisions infrastructure using configuration files
WSL2	Windows Subsystem for Linux 2 — runs Linux on Windows

END OF ENVELOPE A — TECHNICAL PROPOSAL | Team DeployX | DESC-MRD-2026-CNC-088 | June 2026